HW 9 due next Monday.
Quiz 12 coming soon.

Extra Exercise →

Class 26: Cook-Levin Theorem

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Cook-Levin Theorem

Proof

Textbook [TCS] Section 15.3-15.4

https://introtcs.org/public/lec_13_Cook_Levin.html
NP, NP Completeness

Gemini generated cover image.

Recap

$$3\text{SAT}\left(\begin{array}{l}(x_1 \vee x_2 \vee \bar{x}_3)\\ \wedge (x_3 \vee x_2 \vee x_4)\\ \wedge (x_1 \vee x_2 \vee x_3)\end{array}\right) = 1$$

assignment

iff

$$\exists w \text{ s.t. } V(x, w) = 1,$$

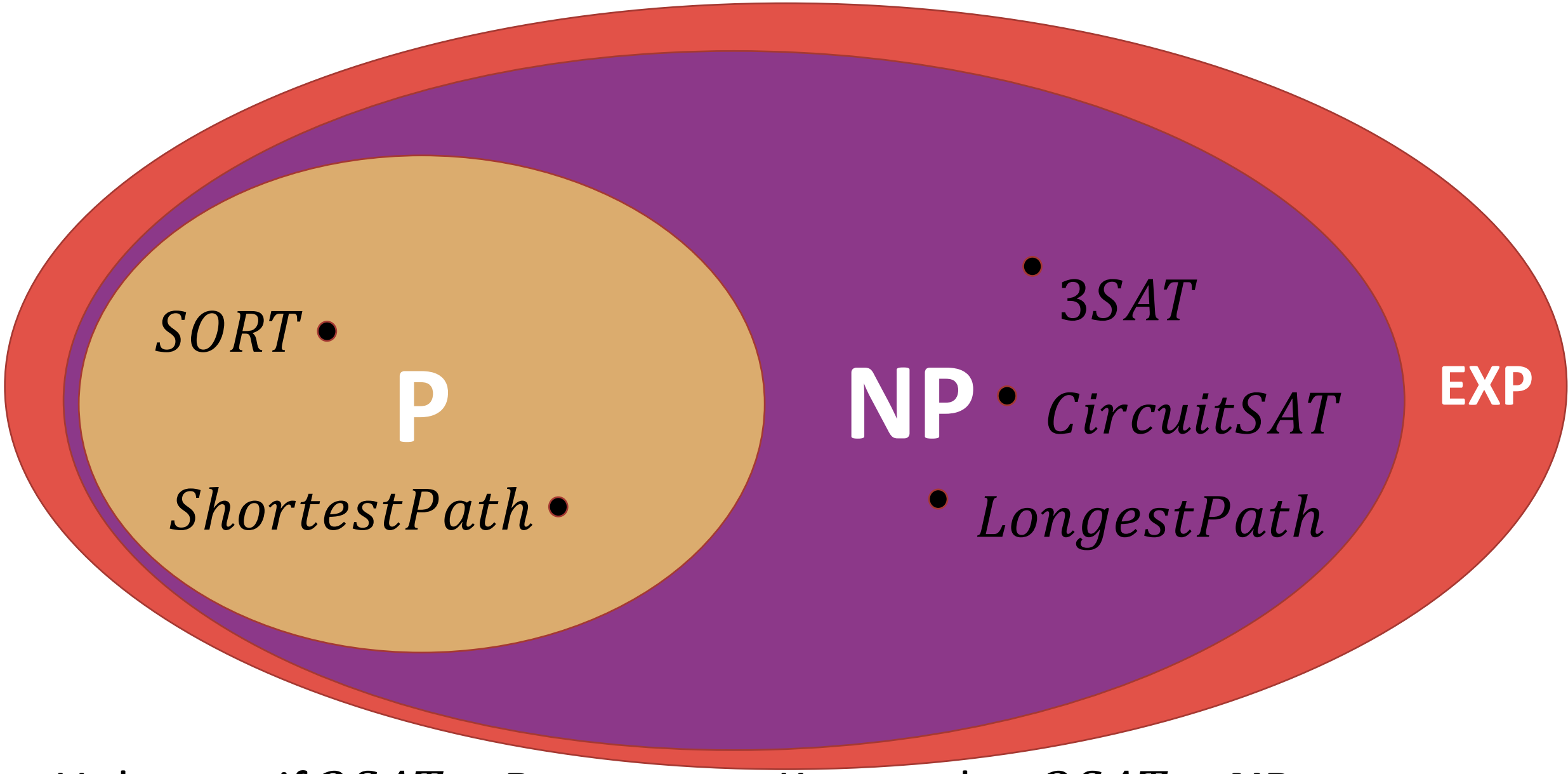
where w is assignment to $x_1, x_2, \ldots$.

The Class NP

Functions that can be **verified** in polynomial time by a standard Turing Machine.

Correctness of a **1** output can be *verified* in polynomial time given a witness.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$ such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n, F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x, w) = 1$.



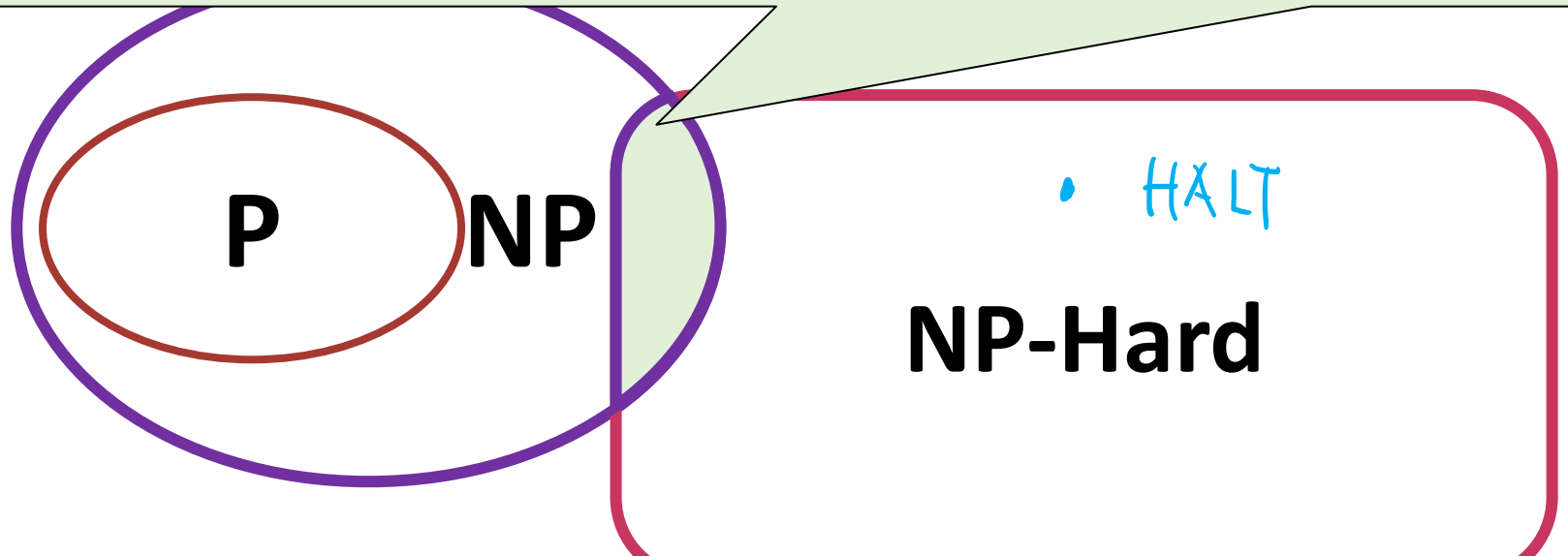
Unknown if $3SAT \in$ **P**

Known that $3SAT \in$ **NP**

NP-Hard and NP-Complete

Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Definition: A function G is **NP-Complete** if $G \in \mathbf{NP}$ and G is **NP-Hard**.



Toward $P \subsetneq NP$
Cook-Levin Theorem

Cook-Levin Theorem

(Theorem 15.6 in the TCS Book):

For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

1st NP-complete

Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Equivalently: **3SAT** is (in) **NP-Hard**

Stephen A. Cook, 1971

The Complexity of Theorem-Proving Procedures

Stephen A. Cook

University of Toronto

Summary

It is shown that any recognition problem solved by a polynomial time-bounded nondeterministic Turing machine can be "reduced" to the problem of determining whether a given propositional formula is a tautology. Here "reduced" means, roughly speaking, that the first problem can be solved deterministically in polynomial time provided an oracle is available for solving the second. From this notion of reducible, polynomial degrees of difficulty are defined, and it is shown that the problem of determining tautologyhood has the same polynomial degree as the problem of determining whether the first of two given graphs is isomorphic to a subgraph of the second. Other examples are discussed. A method of measuring the complexity of proof procedures for the predicate calculus is introduced and discussed.

certain recursive set of strings on this alphabet, and we are interested in the problem of finding a good lower bound on its possible recognition times. We provide no such lower bound here, but theorem 1 will give evidence that {tautologies} is a difficult set to recognize, since many apparently difficult problems can be reduced to determining tautologyhood. By reduced we mean, roughly speaking, that if tautologyhood could be decided instantly (by an "oracle") then these problems could be decided in polynomial time. In order to make this notion precise, we introduce query machines, which are like Turing machines with oracles in [1].

A query machine is a multitape Turing machine with a distinguished tape called the query tape, and three distinguished states called the query state, yes state, and no

Stephen Cook (2015 picture), University of Toronto

ПРОБЛЕМЫ ПЕРЕДАЧИ ИНФОРМАЦИИ

Том IX *1973* *Вып. 3*

Universal Search Problems

УДК 519.14

УНИВЕРСАЛЬНЫЕ ЗАДАЧИ ПЕРЕБОРА

Л. А. Левин

В статье рассматривается несколько известных массовых задач «переборного типа» и доказывается, что эти задачи можно решать лишь за такое время, за которое можно решать вообще любые задачи указанного типа.

После уточнения понятия алгоритма была доказана алгоритмическая неразрешимость ряда классических массовых проблем (например, проблем тождества элементов групп, гомеоморфности многообразий, разрешимости диофантовых уравнений и других). Тем самым был снят вопрос о нахождении практического способа их решения. Однако существование алгоритмов для решения других задач не снимает для них аналогичного вопроса из-за фантастически большого объема работы, предписываемого этими алгоритмами. Такова ситуация с так называемыми переборными задачами: минимизации булевых функций, поиска доказательств ограниченной длины, выяснения изоморфности графов и другими. Все эти задачи решаются тривиальными алгоритмами, состоящими в переборе всех возможностей. Однако эти алгоритмы требуют экспоненциального времени работы и у математиков сложилось убеждение, что

Leonid Levin
(born 1948)
Boston University

3SAT is NP-Complete

Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Definition: A function G is **NP-Complete** if $G \in \mathbf{NP}$ and G is **NP-Hard**.

Cook: **3SAT** is **NP-Hard**

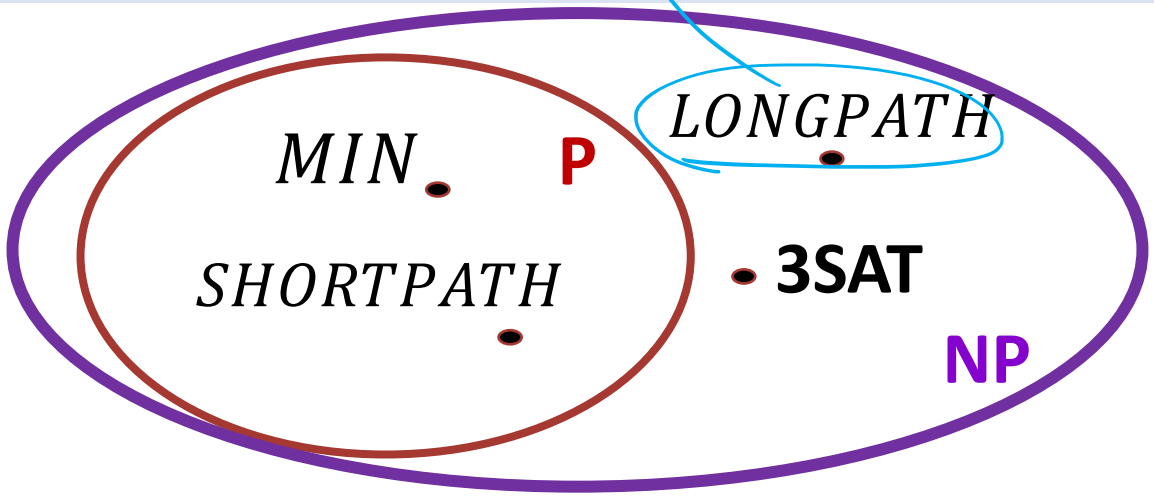
$\Rightarrow$ **3SAT** is **NP-Complete** by **3SAT** $\in$ **NP**

Making Progress on $\mathbf{P} \subsetneq \mathbf{NP}$

Cook-Levin Theorem: For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

Equivalently: 3SAT $\in$ NP-Hard

After showing 3SAT $\in$ NP
$\Rightarrow$ 3SAT $\in$ NP–Complete



If **3SAT $\in$ P**

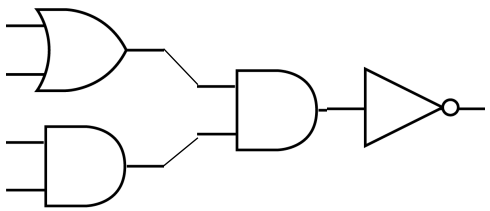
If **3SAT $\notin$ P**

$\mathbf{CircuitSAT} \leq_p \mathbf{NANDSAT} \leq_p \mathbf{3SAT}$

Easy: 3SAT $\leq_p$ **CircuitSAT**

$\mathbf{CircuitSAT} \leq_p$ 3SAT

Suppose we are given Boolean circuit C. How do we transform it into a 3-CNF formula F such that C is satisfiable if and only if F is satisfiable?



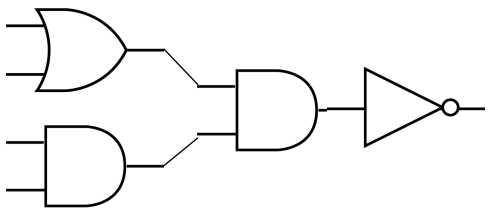
CircuitSAT $\leq_p$ NANDSAT

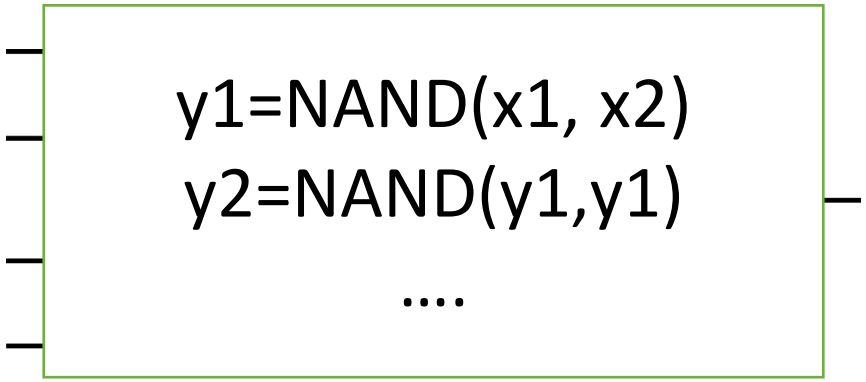
C-SAT (C) = N-SAT (R (C))

Suppose we are given Boolean circuit C. How do we transform it into a NAND-Circuit N such that C is satisfiable if and only if N is satisfiable?

Yes: Substitute AND, OR, NOT with NAND

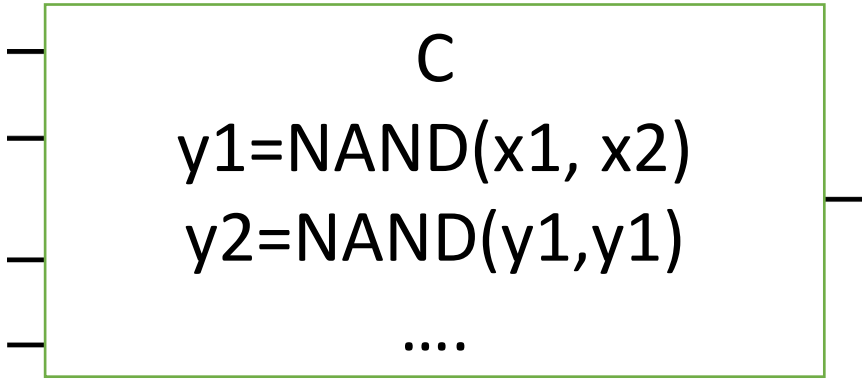
At most 3x more gates than C





$\mathbf{NANDSAT} \leq_p$ 3SAT

Suppose we are given NAND circuit C. How do we transform it into a 3-CNF formula F such that C is satisfiable if and only if F is satisfiable?



$F(\mathrm{x}) = (x_1 \vee x_2 \vee x_3) \wedge (\ldots) \wedge \ldots.$

NANDSAT $\leq_p$ 3SAT

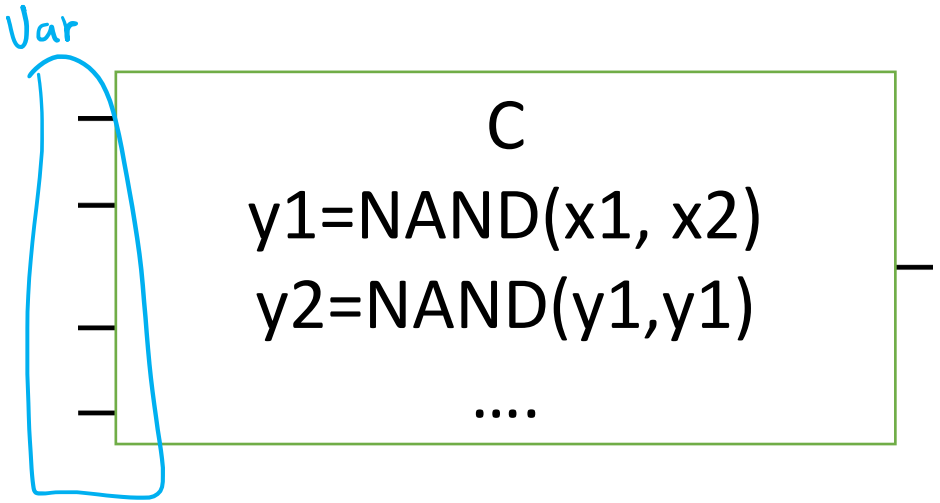
Useful lemma: for every gate $z = NAND(x, y)$, one can write a 3-CNF formula G_{xyz} over them such that $z = NAND(x, y)$ iff G_{xyz} is satisfiable.

$$G_{xyz} = (x \vee y \vee z) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

x	y	z=NAND(x,y)	G_{xyz}
0	0	1	$(x \vee y \vee z)$
0	1	1	$(x \vee \bar{y} \vee z)$
1	0	1	$(\bar{x} \vee y \vee \bar{z})$
1	1	0	$(\bar{x} \vee \bar{y} \vee \bar{z})$
others			

If x = 0 and y = 0, then z must be 1
If x = 0 and y = 1, then z must be 1
If x = 1 and y = 0, then z must be 1
If x = 1 and y = 1, then z must be 0
Otherwise, don't care.

$$G_{xyz} = (x \vee y \vee z) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



$$F(\overbrace{x1,x2,\ldots,}^{\text{inp var}}\overbrace{y1,y2,\ldots}^{\text{internal wires}})$$

$$= G_{x_1 x_2 y_1} \wedge G_{y_1 y_1 y_2} \wedge \ldots.$$

Reduction from **NANDSAT** to 3SAT:
NANDSAT(C) = 3SAT(R (C)), where reduction R(C) = F for all C

Conclusion:

$$\textbf{CircuitSAT} \leq_p \textbf{NANDSAT} \leq_p \text{3SAT}$$

Proof of Cook-Levin Theorem

Proving the Cook-Levin Theorem

Cook-Levin Theorem (Theorem 15.6 in the TCS Book):
For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Proof strategy:
Find a problem Z that is **NP-Hard** and show that
$Z \leq_P 3SAT$.

Is $CircuitSAT$ **NP-Hard** ? YES

We have $\underline{CircuitSAT} \leq_p 3SAT$

We are finished if $CircuitSAT$ is also NP-Hard (why?)

CircuitSAT

Input: C, a string representing an n-input Boolean circuit.

Output: **1** iff there exists a string $x \in \{0,1\}^n$ such that $C(x) = 1$.

How to prove $CircuitSAT$ is **NP-Hard** ?

CircuitSAT

Input: C, a string representing an n-input Boolean circuit.

Output: 1 iff there exists a string $x \in \{0,1\}^n$ such that $C(x) = 1$.

Definition: A Boolean function G is **NP-Hard** if every $F \in \mathbf{NP}$ can be reduced to G: $F \leq_p G$.

Proving $CircuitSAT$ is NP-*Hard*

Definition of Polynomial-Time Reduction ($\leq_p$): For any $F, G\colon \{0,1\}^* \to \{0,1\}$. $F \leq_p G$ if there is a polynomial-time computable $R\colon \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*, F(x) = G(R(x))$.

$$\forall F \in \mathbf{NP}\colon F \leq_p \underbrace{CircuitSAT}_{\text{ckt}}$$

$$\forall F \in \mathbf{NP}\colon \exists R \in \mathbf{P}\colon \forall x \in \{0,1\}^*\colon F(x) = CircuitSAT(R(x))$$

We need to show that R exists, for <u>every</u> function in **NP**!

Recall the Definition of Class NP

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Definition of $\leq_p$: For any F, G: $\{0,1\}^* \to \{0,1\}$.
$F \leq_p G$ if there is a p-time $R: \{0,1\}^* \to \{0,1\}^*$
such that for every $x \in \{0,1\}^*, F(x) = G(R(x))$.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Goal: $\forall F \in \mathbf{NP}$: $F \leq_p CircuitSAT$

$$\forall F \in \mathbf{NP}: \exists R \in \mathbf{P}: \forall x \in \{0,1\}^*: F(x) = CircuitSAT(R(x))$$

Definition of $\leq_p$: For any F, G: $\{0,1\}^* \to \{0,1\}$.
$F \leq_p G$ if there is a p-time $R: \{0,1\}^* \to \{0,1\}^*$
such that for every $x \in \{0,1\}^*, F(x) = G(R(x))$.

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Goal: $\forall F \in \mathbf{NP}$: $F \leq_P CircuitSAT$

$$\forall F \in \mathbf{NP}: \exists R \in \mathbf{P}: \forall x \in \{0,1\}^*: F(x) = CircuitSAT(R(x))$$

ckt hardwires x

There exists a Boolean circuit $C_x(\star)$ that computes $V(x, \star)$:

Input: $\star \in \{0,1\}^{n^a}$ length = n^a

Output: $V(x, \star)$

How do we know C_x exists?

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

There exists a Boolean circuit $C_x(\star)$ that computes $V(x, \star)$:

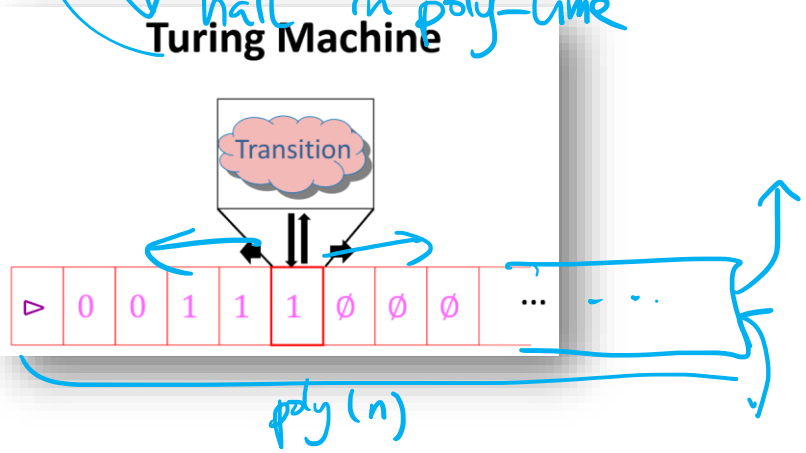
Input: $\star \in \{0,1\}^{n^a}$ length = n^a

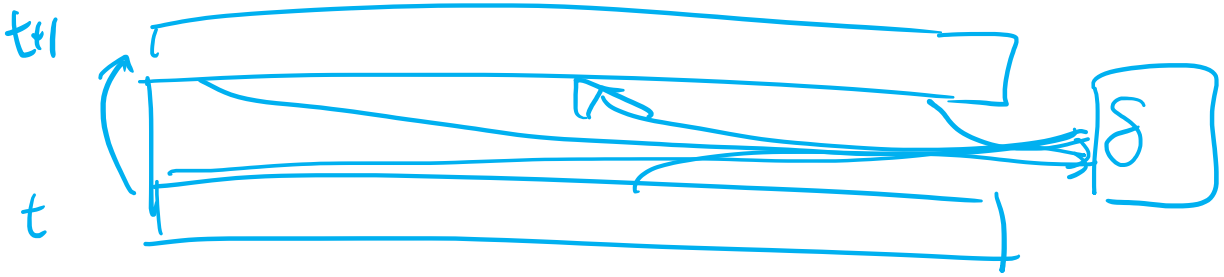
Output: $V(x, \star)$

Boolean circuit $C_x(\star)$ that computes $V(x,\star)$

$V \in \mathbf{P}$ so we have TM M computes V in poly-time.
Fixing x, we also have M_x computes $V(x,\ \star)$ in poly-time.
We want C_x that <u>simulates</u> M_x and $|\ C_x\ | = poly(|x|)$.

halt in poly-time

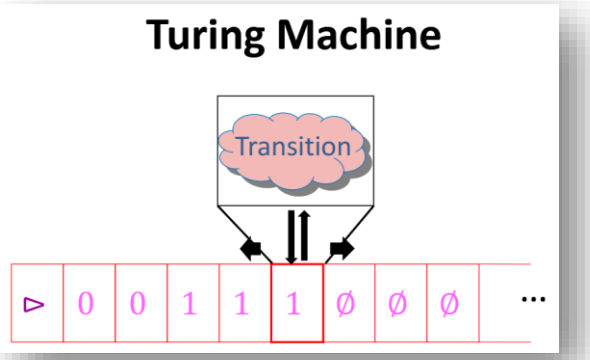




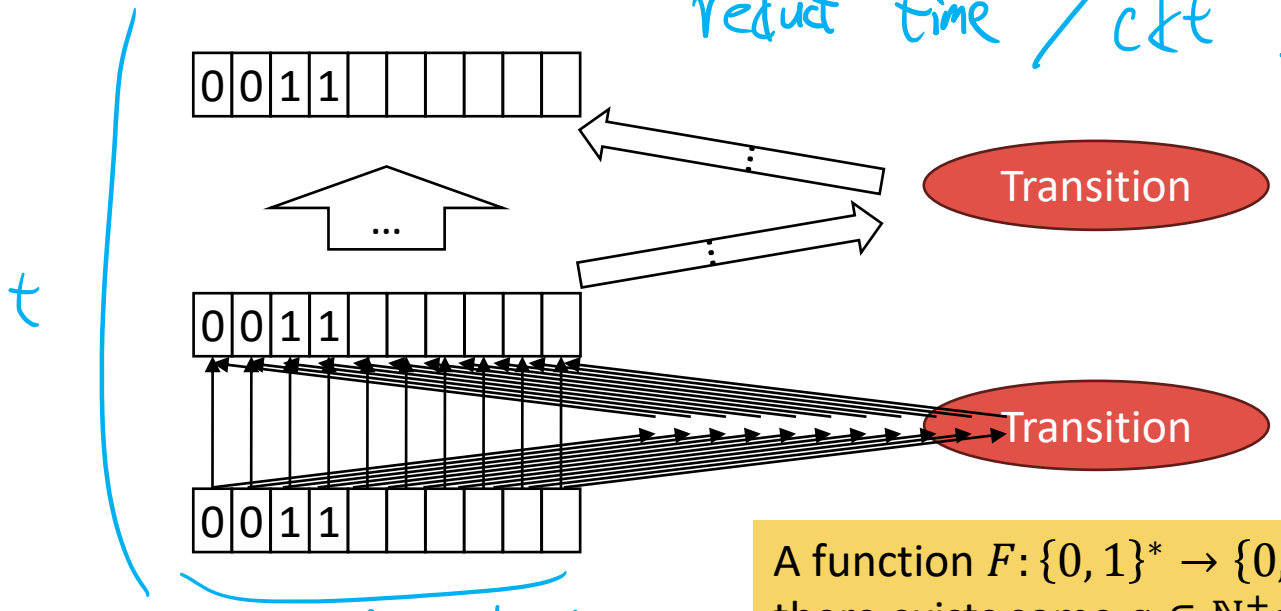
A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Boolean circuit $C_x(\star)$ that computes $V(x,\star)$

$V \in \mathbf{P}$ so we have TM M computes V in poly-time.
Fixing x, we also have M_x computes $V(x,\ \star)$ in poly-time.
We want C_x that <u>simulates</u> M_x and $|\ C_x\ | = poly(|x|)$.



reduct time / ckt size $= O(t \cdot \ell)$



$t, \ell = poly(n)$
$|x| = n$

$\ell = poly(n) \geq n + n^a$

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$ and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Boolean circuit $C_x(\star)$ that computes $V(x,\star)$

$V \in \mathbf{P}$ so we have TM M computes V in poly-time.
Fixing x, we also have M_x computes $V(x,\ \star)$ in poly-time.
We want C_x that <u>simulates</u> M_x and $|\ C_x\ | = poly(|x|)$.

Theorem:
For any $f \in \mathbf{P}$, (f is computable by <u>TM</u> in poly time)
$f \in \mathbf{P}_{/\mathbf{poly}} = \bigcup_{c\in\mathbb{N}} SIZE(n^c)$ (f is computable by <u>circuit</u> in poly size)

Theorem 13.12 (Non-uniform computation contains uniform computation)
There is some $a \in \mathbb{N}$ s.t. for every nice $T : \mathbb{N} \to \mathbb{N}$ and $F : \{0,1\}^* \to \{0,1\}$,

$$TIME(T(n)) \subseteq SIZE(T(n)^a)\ .$$

Completing the Proof

Cook-Levin Theorem: For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

We have proven *CircuitSAT* is **NP-Hard**:

$$\forall F \in \mathbf{NP}: F \leq_p CircuitSAT$$

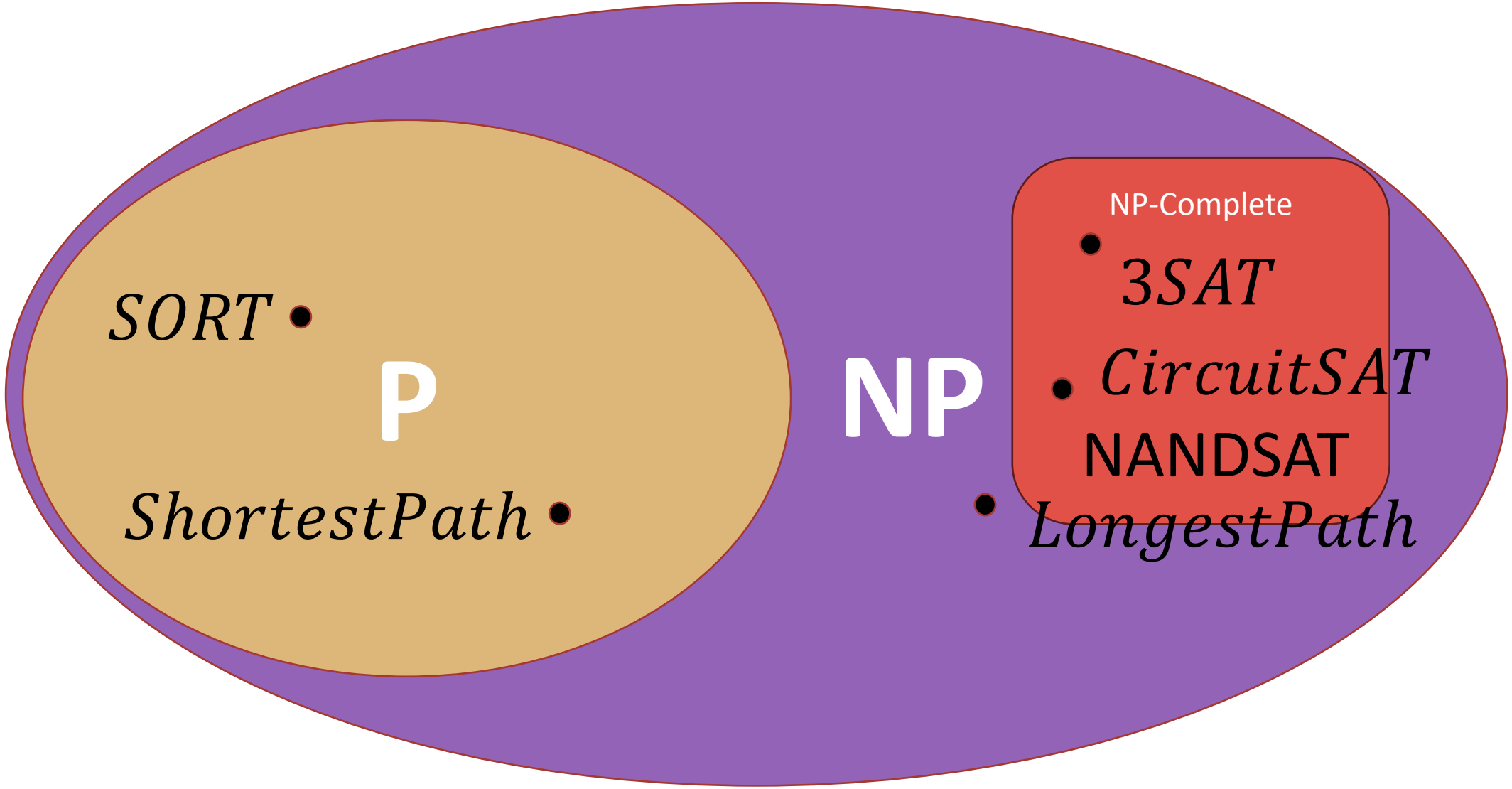
Cook-Levin Theorem: For every $F \in \mathbf{NP}$, $F \leq_p$ **3SAT.**

We have proven *CircuitSAT* is **NP-Hard**:

$$\forall F \in \mathbf{NP}: F \leq_p CircuitSAT$$

$$F \leq_p CircuitSAT \leq_p 3SAT$$

Thus, if there is a polynomial time algorithm for 3SAT, there is a polynomial time algorithm for *every* problem in **NP**!



Unknown if $3SAT \in$ **P**

Known that $3SAT \in$ **NP**

Alternate Definition: Nondeterministic Turing Machines

Standard TM Definition

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

How should we define a ***Nondeterministic Turing Machine****?*

Nondeterminstic TM

A **Nondeterministic** *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \to Pow([C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\})$$

Deterministic (Standard) TM Execution

Definition. The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.

   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize two natural number variables, $i = 0, s = 0$.
3. **repeat**
   1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
   2. $s := s', T[i] := \sigma'$
   3. if D = $\mathbf{R}$: $i := i + 1$

      if D = $\mathbf{L}$: $i := \max\{i - 1, 0\}$

      if D = $\mathbf{H}$: **break**

*How should we define a **Nondeterministic Turing Machine** execution?*

Nondeterministic TM Execution

Definition. The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.
   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize a set of configurations, $Z = \{(T, i = 0, s = 0)\}$
3. **repeat**
   - $Z' = \{\}$
   - **foreach** $(T, i, s) \in Z$:
     - **foreach** $(s', \sigma', D) \in \boldsymbol{\delta}(s, T[i])$
       - 2. $T' = T, T'[i] := \sigma'$
       - 3. if D = $\mathbf{R}$: $i' := i + 1$
         - if D = $\mathbf{L}$: $i' := \max\{i - 1, 0\}$
         - if D = $\mathbf{H}$: **break** (outer loop)
       - $Z' = Z' \cup \{(T', i', s')\}$
   - $Z = Z'$

Nondeterministic TM Execution

Definition. The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.
   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize a set of configurations, $Z = \{(T, i = 0, s = 0)\}$
3. **repeat**
   - $Z' = \{\}$
   - **foreach** $(T, i, s) \in Z$:
     - **foreach** $(s', \sigma', D) \in \boldsymbol{\delta}(s, T[i])$
       - 2. $T' = T, T'[i] := \sigma'$
       - 3. if D = $\mathbf{R}$: $i' := i + 1$
         - if D = $\mathbf{L}$: $i' := \max\{i - 1, 0\}$
         - if D = $\mathbf{H}$: **break** (outer loop)
       - $Z' = Z' \cup \{(T', i', s')\}$
   - $Z = Z'$

4. If the process finishes (the repeat breaks), the result of this process is $M(\cdot) = T'[1], \ldots, T'[m_r]$ where m_r is the smallest integer such that $\forall z > m_r. T'[z] = \emptyset$. Otherwise, $M(\cdot) = \perp$.

Nondeterministic TM Execution

Definition. The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

1. Initialize T as $\triangleright, x_0, x_1, \ldots, x_{n-1}, \emptyset, \emptyset, \emptyset, \ldots$ where $n = |x|$.
   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize a set of configurations, $Z = \{(T, i = 0, s = 0)\}$
3. **repeat**
   $Z' = \{\}$
   foreach $(T, i, s) \in Z$:
   foreach $(s', \sigma', D) \in \boldsymbol{\delta}(s, T[i])$
   2. $T' = T, T'[i] := \sigma'$
   3. if $D = \mathbf{R}$: $i' := i + 1$
   if $D = \mathbf{L}$: $i' := \max\{i - 1, 0\}$

There are lots of other (better) ways to define the output of a nondeterministic TM, such as if any execution halts and outputs "1" the output is "1".

   $Z = Z'$

4. If the process finishes (the repeat breaks), the result of this process is $M(\cdot) = T'[1], \ldots, T'[m_r]$ where m_r is the smallest integer such that $\forall z > m_r. T'[z] = \emptyset$. Otherwise, $M(\cdot) = \perp$.

Alternative Definition of NP

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if there exists a <u>non-deterministic</u> Turing machine M and some constant $a \in \mathbb{N}^+$ such that $M(x)$ computes $F(x)$ in NTIME(n^a) for all n-bit input x.

NTIME: number of transitions from the initial to the halting state.

Equivalent to:

A function $F: \{0,1\}^* \to \{0,1\}$ is in **NP** if
there exists some $a \in \mathbb{N}^+$and $V: \{0,1\}^* \to \{0,1\}$
such that $V \in \mathbf{P}$ and $\forall x \in \{0,1\}^n$,
$F(x) = 1 \Leftrightarrow \exists w \in \{0,1\}^{n^a}$ such that $V(x,w) = 1$.

Logistics

HW 9 due next Monday.
Quiz 12 coming soon.

Plan

More NP-Complete Problems

What if P $\neq$ NP?

What if P = NP?

Textbook [TCS] Section 16

https://introtcs.org/public/lec_14_PvsNP.html
What if P equals NP?